01 / IMPLEMENTACIÓN Y PRUEBAS

De una consulta vulnerable a una consulta segura

Corrección técnica del formulario de acceso con Node.js, Express y SQLite

Alan Flores Denegri | 26 de octubre de 2026

RESUMEN

El formulario de la práctica aceptaba un usuario y una contraseña, pero el servidor insertaba ambos valores directamente en el texto SQL. Eso permitía cambiar la condición de búsqueda. La corrección sustituye esa interpolación por dos parámetros y conserva el funcionamiento de los usuarios legítimos.

01 Objetivo y entorno

Objetivo: impedir que el contenido de los campos altere la consulta SQL y confirmar que el inicio de sesión normal siga disponible.

La aplicación sirve un archivo index.html con Express. El formulario envía un POST a /login-vulnerable. SQLite usa una base en memoria con las cuentas de demostración admin/admin y juan/1234. El servidor se ejecuta en 127.0.0.1:3000 para esta práctica local.

02 Causa del fallo

La versión inicial construía una sola cadena con el SQL y los valores recibidos. Al introducir una comilla, el valor podía cerrar el literal; con OR 1=1 y un comentario --, la comprobación de la contraseña quedaba anulada.

CONSULTA ORIGINAL / REPRESENTACIÓN DEL FALLO

```
const query = `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`;
db.get(query, (err, row) => { ... });
```

03 Cambio aplicado

Los signos ? fijan los lugares donde SQLite incorporará los valores. El arreglo [username, password] se entrega por separado a db.get. Así, el contenido del formulario se compara como dato literal y no modifica la estructura de la consulta.

Se conservó la ruta del POST para que el formulario siguiera funcionando. El título visible del formulario se actualizó a «Login (consulta parametrizada)».

04 Verificación

Se enviaron peticiones al servidor ya corregido. El código 200 confirma accesos válidos; el 401 indica que no se encontró una combinación de credenciales.

CONSULTA PARAMETRIZADA / COMPARACIÓN

```
const query = "SELECT * FROM users WHERE username = ? AND password = ?";
db.get(query, [username, password], (err, row) => { ... });
```

Caso	Resultado tras el cambio
admin / admin	200 - admin
juan / 1234	200 - juan
admin'-- / z	401 - rechazado
' OR 1=1 -- / x	401 - rechazado

Conclusión

La corrección elimina la vía de inyección comprobada y mantiene el comportamiento esperado para credenciales válidas. Quedan fuera de este cambio el almacenamiento seguro de contraseñas, el límite de intentos y la gestión de sesiones; serían tareas necesarias antes de usar el sistema fuera del laboratorio.

Fuente: reporte de práctica proporcionado por el usuario y verificación local del proyecto SeguridadInformatica (login/index.html y login/server-vulnerable.js).

EVIDENCIA VISUAL / ALAN FLORES DENEGRI

Capturas de la práctica

Las dos capturas muestran el mismo intento de inyección antes y después de parametrizar la consulta.

01 Antes de la corrección



El texto de inyección con contraseña «x» obtuvo ok: true y devolvió admin.

02 Después de la corrección



La consulta parametrizada rechazó el mismo intento: ok: false y «Credenciales inválidas».

Capturas del navegador proporcionadas por el usuario durante la práctica.